

DESIGN ITERATION LOG · 3-LENS SCORING → REBUILD

VEMO 设计迭代日志

以 Wildpanda 为起点,循环 3 次、每次切换工程师视角打分,收敛出一套「给开发者加速」的治理框架

基线 Wildpanda 6.1/10

终值 VEMO 8.6/10

北极星 time-to-correct-commit ↓

产物 VEMO/ 可落地框架

M

方法:为什么是「3 视角循环」

单一视角会自我锚定。我让三类工程师轮流上场,各自带着不同的评分坐标系与偏见来挑刺;每一轮的批评直接转化为对设计的改动,下一轮换人复盘。三轮之后,设计在三套坐标系下都站得住,才收敛为 VEMO。

迭代	视角(工程师人格)	评分坐标系	它最在乎
①	Claude Code 平台工程师	机制层 / 上下文工程 / 原语归位	「该是 hook 的别写成提示词」
②	OpenAI / Spec-Kit + DX	开发者速度 / 规范严谨 / 模型路由 / 评测	「没人用的框架=废框架」
③	Staff 工程师 / SRE(对抗者)	失效模式 / 可采纳 / 成本 / 可观测 / 跨平台	「周五晚上累瘫的人还会用它吗」

分数轨迹(每轮镜头下的平均分)



注:迭代①②的「后」分是该镜头自有坐标系下的平均;迭代③故意用对抗坐标系重评 v2,所以分数先回落(暴露鲁棒性缺口)再被修复;终值是用最初分析报告的 11 维统一复评,与 Wildpanda 6.1 可比。

① 迭代一 · Claude Code 平台工程师视角



PERSPECTIVE 1 / ANTHROPIC HARNESS

「我用原语思考」

CLAUDE.md/AGENTS.md · MCP · skills · subagents · hooks · plan mode。信条:把约束放进正确的层,确定性机制永远胜过提示词。^[1]

🚫 这个镜头看 WILDPANDA

最致命的错位:该做成 hook 的约束,全被写成了 prose。~30 条「hard gate」依赖模型读到并照做——而 hook「执行确定性代码,不会幻觉」。其次:开局 20 步全读把约束推进「失忆区」^[3];subagent 仅起步;skills 用得不错。

本镜头坐标系打分(Wildpanda → 迭代①设计)

评分轴	Wildpanda → v1	说明
强制确定性(hook vs prose)	3 → 9	新增 <code>enforcement/</code> :hooks + CI 兜底 + validator
原语归位(约束放对层)	5 → 9	safety → hook,advice → spec,verify → subagent
上下文纪律(JIT vs 前置)	4 → 8	<code>specs/_manifest.yaml</code> 按需加载 + 瘦 AGENTS.md
子代理隔离/验证	5 → 8	新增 <code>agents/governance-judge.md</code>

镜头均分 5.0 → 8.6 (+3.6)

✓ 本轮落地到 VEMO 的改动

- `enforcement/hooks/` — PreToolUse 拦越界编辑(`guard-scope.sh`)、破坏性命令、密钥;Stop 提醒未验收。
- `enforcement/validators/task_state.py` — 把任务态做成机器可读 `front-matter`,hook/CI 确定性解析,不再靠猜 `prose`。
- `enforcement/ci/pre-commit` — 服务端兜底, `--no-verify` 绕不过。
- 瘦 `AGENTS.md` (~50 行路由)+ `specs/_manifest.yaml` JIT 加载。

② 迭代二 · OpenAI / Spec-Kit + 开发者体验视角



PERSPECTIVE 2 / SPEC-DRIVEN + DX

「速度即产品」

给百万开发者发 `spec-driven` 工具。死磕 `time-to-first-correct-commit`、摩擦、模型路由、评测。^[2]

⊖ 这个镜头看「迭代①的 v1」

机制补上了,但**速度还没救**:小改动和核心改动走一样重的流程;模型被硬编码成 `opus(agents/*.md)`,不随能力调档;没有任何评测能证明「框架真被遵守 / 能不能减仪式」。规范虽好但缺 EARS 式可测标准。

本镜头坐标系打分(v1 → 迭代②设计)

评分轴

v1 → v2

说明

开发者速度(time-to-commit)	4 → 9	风险分层 R0/R1/R2;R0 快路径 ~3 步
仪式与风险成正比	3 → 9	ceremony matrix 按 tier×risk 缩放
模型路由/能力适配	2 → 9	<code>capability.tier</code> + <code>model_routing</code> (对标 Kiro Auto router)
规范严谨(可测/EARS)	6 → 9	<code>verify.spec</code> 强制 EARS 可证伪标准
可度量 / 评测	2 → 8	新增 <code>eval/</code> 一致性套件 + 跨模型 A/B
标准对齐 (AGENTS.md) ^[4]	7 → 9	瘦身到「简洁优先」标准形态

镜头均分 4.0 → 8.8 (+4.8)

✓ 本轮落地到 VEMO 的改动

- `vemo.config.yaml` → `risk_tiers` — R0 快路径(典型 docs/config),仪式≈Wildpanda 的 1/3。
- `capability.tier` + `model_routing` — 升级模型只调一个档,仪式自动变轻;按阶段路由 opus/sonnet/haiku。
- `specs/verify.spec.md` — EARS 式「WHEN...SHALL...」可测标准。
- `eval/` — 场景化一致性套件,指标含 `r0_steps`、`ceremony_overhead`、`judge_catch_rate`。

③ 迭代三 · Staff 工程师 / SRE(对抗者)视角



PERSPECTIVE 3 / PRODUCTION ADVERSARY

「我专找它怎么死」

见过太多框架死在生产。我攻击:hook 怎么绕?爆炸半径多大?累瘫的人还会用吗?什么会漂移?judge 多烧钱?

⊖ 这个镜头用对抗坐标系重评「V2」(分数故意先回落,暴露缺口)

客户端 hook 可被 `--no-verify` 绕过;judge 每次 R1 都跑会很烧钱也拖慢;一个 config 虽好但若复杂也劝退;PowerShell 只能 Windows(本机是 Linux);规范仍可能与代码漂移;工具缺失时若静默放行=假安全。

本镜头坐标系打分(v2 → 迭代③加固)

评分轴	v2 → v3		加固手段
强制鲁棒(抗绕过)	6	→ 9	CI 服务端为权威兜底;校验 hook/CI 配置未被篡改
可采纳(累瘫也愿用)	6	→ 9	R0 快路径 + 单一 config + 一键 <code>install.sh</code>
成本控制 (judge/token)	5	→ 8	judge 只在所需 tier 跑;high-tier 下 R1 自验证
可维护 / 抗漂移	6	→ 8	每个 spec 行数上限 + 漂移检测 CI + manifest
可观测	4	→ 8	<code>telemetry: gate_events</code> 记录门命中/缺失
跨平台	4	→ 9	bash install + stdlib python;去 PowerShell 耦合

镜头均分 5.2 → 8.5 (+3.3)

✓ 本轮落地到 VEMO 的改动

- 纵深防御:hook 给快反馈, `enforcement/ci/pre-commit` 做服务端权威,二者缺一仍有兜底。
- 优雅降级: `degrade_gracefully` — 工具缺失时「响亮告警 + 记日志」,绝不静默放行。
- 成本守门:judge 仅按 risk tier 触发;high-tier 下 R1 自验证,不滥用大模型。
- 可移植引导 + 遥测 + 漂移检测,以及每个 spec 的反膨胀行数上限。

三轮之后,设计在「机制 / 速度 / 鲁棒」三套坐标系下都站得住。用最初分析的 11 维统一复评,与 Wildpanda 6.1 直接可比:

维度	Wildpanda	VEMO	关键改动
设计哲学	9	9	保留四原则
持久化/无状态归约	8	9	机器可读 front-matter
多会话并发安全	9	9	保留 + 加 CI 校验所有权
人在环控制	9	9	保留,只在 R2/不可逆设门
强制力(机制 vs 文字)	3	9	hooks + CI 兜底 + validator
上下文工程	4	8	JIT manifest + 瘦 AGENTS.md
复用性与解耦	8	8	保留 template/instance,单 config
可观测性与评测	2	8	eval 套件 + 遥测
模型能力适配	3	9	capability.tier + model_routing
标准对齐	7	9	瘦 AGENTS.md + hooks/subagents
规范可维护性	5	8	行数上限 + 漂移检测
综合	6.1	8.6	+2.5

北极星:对开发者的加速(可度量)

指标	Wildpanda	VEMO
低风险改动(R0)的治理步骤	同一重流程	~3 步(≈60–70% 仪式)
机器强制的不可逾越红线	0(全靠自觉)	5 条(hook+CI)
每会话上下文前置成本	~20 份规范全读	1 份(safety)+ 按需
模型升级(4.6 → 4.8)的适配动作	手动重调散落规则	改一个 tier ,仪式自动变轻
「框架是否被遵守」可否度量	否	是(eval + telemetry)

保留 / 改造 / 新增 / 砍掉(相对 Wildpanda)

✓ 保留

- 证据高于断言 · 库即事实源
- 多会话 chat_id/心跳/接管
- template/instance/runtime 三分
- plan-first · PR-only · 人在环
- 产物溯源(coding \$provenance)

↻ 改造

- 20 步全读 → JIT manifest
- prose 硬门 → hooks/CI 机制门
- 统一重流程 → R0/R1/R2 分层
- 硬编码 opus → capability.tier+路由
- 自报门通过 → judge 独立翻牌

+ 新增

- **enforcement/** 机制层(本质增量)
- 机器可读任务 front-matter
- **eval/** 一致性评测 + 遥测
- governance-judge 独立验证者
- 跨平台 install.sh

- 砍掉/降级

- 「Agents loading/loaded」口令
- 重叠多重 prose 门(并入机制)
- 引用未落地原语的触发器
- 小任务的强制重流程
- PowerShell-only 路径

◎ 一句话

Wildpanda 想清了「治什么」;VEMO 把它做成了**机制保证、数据可证、且随模型变强而自动变轻**的东西。治理的终局不是规则更多,而是**恰到好处地少**——而「少」的前提,是把不可让步的那几条交给机器,

而不是交给运气。

Sources / 参考最佳实践: [1] Claude Code hooks/subagents/skills(2026) · [2] Spec-driven dev / Kiro Auto router(2026) · [3] 12-Factor Agents · [4] AGENTS.md 标准 · Anthropic: context engineering

VEMO · 设计迭代日志 · 生成于 2026-06-16 · 作者视角: Claude (Opus 4.8) 扮演平台工程师

框架文件见 VEMO/README.md · 原始分析见 doc/wildpanda-governance-analysis.html